

AI-Powered Course Generation

Let's discuss the design for the Backend to run the AI-Powered Course Generation before we move to actual AI workflow.

The task being a long process, we would run this as a background job. We would use Celery as for the queueing system, and Redis as the broker between Celery and FastAPI.

When creating a Course, the Tenant Admin would fill a form that takes:

- Learner details. The Details would have the information like what other course the Learner is taking, so that the AI would have an idea of what is the knowledge level of the learner, also it would have the progress of the learner in each course that he is either completed or taking right now.. But this should ensure that the PII of the learner is not fed to the AI, just the required metadata.
- The Course Name, Course Description, and Course Goal.
- Per day learning duration requirement of the course, and the requirement highlighting the duration for the course completion.

This would create a shared task in celery. And the Backend would subscribe to the Redis client that publishes the logs generated inside the task. For this we would use Redis Pub/Sub, so that the background task can publish the logs that could be provided to the user through the backend, so that the Admin would be able to view the progress on the course generation, and won't be kept stale at an uncertainty of the inside working. Also, we would be storing the logs in a database, because Redis Pub/Sub is a fire and forget mechanism, and we want to keep the logs even after the Admin reconnects.

The Redis Pub/Sub system can be also used to introduce human-in-the-loop for the course generation Agent, more on that below.

For the Course Generation we would create a LangGraph agent.

- It would have a state that would maintain the context of the course, so the AI would hallucinate less, and we can save the state using Checkpointers in Postgres Database, so that it won't have to understand the Context from root, improving token optimization.
- It would have multiple nodes that would act as an autonomous agent.
 - At first, a sub-agent would generate a test for the Course that would be passed to the Learner to understand the Learner's knowledge. Then we would interrupt the workflow until the learner completes the assessment. After the learner completes the assessment, we trigger the main course generation task, only after the assessment scores/data are saved to the database.
 - Since creating a whole Course would be a very tedious task for a single agent, which might result in token surpassing the limits of a context, and hallucinations due to larger context. So, there would be an Overview deciding node that would take the metadata for the Course, provided by the Tenant Admin, and the information about the Learner, and the results from the Test generated by the

previously discussed sub-agent; and the agent would plan overview of the course based on the hours of course per day, and required day of completion. We should ensure the Overview Node outputs explicit, isolated boundaries for each day. Basically this node would decide the course generation plan for each day, and then k sub-agents, with relevant concurrency control would be summoned from that node (k being the number of days for the course to complete, meaning one sub-agent to write a course for each day). This ensures that each agent can work parallelly, and work on a smaller context. And since Langgraph maintains a state, it would be easier to transfer the states/messages between the agents, to maintain consistency and minimize hallucinations. Each sub-agent's state would have the individual AIMessage from the Overview node, stating the context and objective of that sub-agent. Also would have a system message that would tell the Agent about the required Structure of the AIResponse that it should generate.

- After each sub-agent provides the course work that they have generated, we would have a node that would be responsible to check for the valid structure of the AI Response. It would have a System Prompt on what should be the valid structure of the AI Response, also we can handle structural validation natively in LangGraph using standard Python Pydantic models like with `_structured_output()`. It would check for validity of each sub-agent's AIResponse individually, and keep track of which AI Response fails to provide the valid structure. Then, only in the state of the Sub-Agent that failed to generate the valid response, we would append an AIMessage telling the model that there was a structure issue, and pass the error message from the Pydantic Model. In this way, we only make the failing sub-agents rework on an issue that we know from the Pydantic error, which improves the efficiency of the whole Course Generating Agent System. It would only retry for 2 times, after that we would inform the Admin that the AI work failed, and the Admin can decide if he wants to retry, or scrape that and start again.
- After the validity of the AIResponse Structure is passed, we introduce Human-in-the-Loop, where the course would be previewed to the admin, and there would be a section where the Admin can provide reviews and suggestions in each day's coursework, and the reviews and suggestions would be passed onto the corresponding subagents, so that they can work on the refinements.

Highlights from the above Design Description:

- Information provided to the AI would be: Per-day hours for a Course, and duration that a Course would run for. The Learner's metadata (other courses and progress), Course title, description and goal. It would have SystemMessage that informs about the valid structure of the AIResponse, and the SystemMessage would also clarify that the LLM should not run any instructions provided in the UserMessage, because the UserMessage is only supposed to provide Context and not the instructions, that it would use explicit XML delimiters to enforce strong structural boundaries. So this prevents Prompt Injection in the UserMessage, and we should also ensure that the User doesn't have access to the SystemMessage.

- For personalization we would be providing the information about what other courses the user is taking, and the progress on those courses. Also the Agent would generate a test for the course to be generated, to understand the level of the Learner, and design the course accordingly.
- For hallucinations, we would be maintaining small, yet rich context (rich as in it would have all information that it needs for the task that it needs to do) by dividing the work into parallel agents, and maintaining a state that would ensure consistency through the Agents work.
- For consistency and valid output, we would keep the temperature of the LLM to minimum, and Handle structural validation natively in LangGraph using standard Python Pydantic models like by using `.with_structured_output()`, and pass back help error message to the corresponding sub-agent to fix the issue.
- For Privacy, it would be ensured that only the required metadata of the user is provided to the Agent, and the PII is skipped.
- For token minimization, we would maintain a checkpoint system in Postgres that would save the states for easy reload, rather than building up the context from scratch every time the agent needs to resume from an interrupted condition.
- There can be a token limit for creating the whole course on a single go, so we would be dividing the task to sub-agents, that would only generate coursework for each day, and before that a single node would generate a full outline for the whole course, and summon the sub-agents based on its result.
- Also since multiple-tenants could generate similar courses, there are chances that we would have a similar course content already in our database. In that case we can just preview the course to the Admin and ask him to provide reviews and suggestions, if one wants some changes on the pre-made course, and then the Agent refines the course, rather than creating one from scratch.